Software License Usage Reporting & Cost Dashboards (Power BI)

Implementation Solution Guide

Version 1.0.0 · 2026-06-29

Contents

Executive Summary — Software License Usage Reporting & Cost Dashboards (Power BI) 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Software License Usage Reporting & Cost Dashboards (Power BI) 4

What this reporting layer is . . . 5

Why it matters . . . 5

How it works . . . 5

The license KPI set (what the dashboards report) . . . 5

Data sources (inputs to the model) . . . 6

What you can control . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 6

2. Requirements Guide — Software License Usage Reporting & Cost Dashboards (Power BI) 6

2.1 Power BI Desktop . . . 6

2.2 Power BI service workspace . . . 7

2.3 Excel with Power Query . . . 7

2.4 Access to license entitlement data . . . 7

2.5 Access to usage / deployment data . . . 7

2.6 Microsoft Graph PowerShell (for the scripts) . . . 7

2.7 Read scopes for license & usage data . . . 7

2.8 On-premises data gateway (only if a source is on-prem) . . . 8

2.9 A reporting/distribution destination . . . 8

Checklist . . . 8

3. Implementation Guide — Software License Usage Reporting & Cost Dashboards (Power BI) 8

Step 1 — Define and record the KPI set . . . 8

Step 2 — Stage the license/entitlement data (Excel Power Query) . . . 9

Step 3 — Generate the usage/deployment extract . . . 9

Step 4 — Build the dataset in Power BI Desktop . . . 9

Step 5 — Author the KPI measures (DAX) . . . 9

Step 6 — Build the dashboard pages . . . 10

Step 7 — Publish to the service . . . 10

Step 8 — Configure scheduled refresh . . . 10

Step 9 — Pilot on a single product family . . . 10

Step 10 — Validate (reconciliation gate) . . . 11
Step 11 — Distribute and expand . . . 11
5. Testing Guide — Software License Usage Reporting & Cost Dashboards (Power BI) 11
Test 1 — Sources connect and refresh . . . 11
Test 2 — KPI set matches definitions . . . 11
Test 3 — Numbers reconcile to source (the gate) . . . 11
Test 4 — Expirations timeline is correct . . . 11
Test 5 — Reharvesting opportunities are credible . . . 12
Test 6 — Scheduled refresh runs unattended . . . 12
Test 7 — Distribution reaches the right audience . . . 12
Pass/fail gate . . . 12
6. Troubleshooting Guide — Software License Usage Reporting & Cost Dashboards (Power BI)
A. Scheduled refresh fails . . . 12
B. Numbers do not reconcile (gate fails) . . . 13
C. KPI values look wrong . . . 13
D. Expirations timeline empty or wrong . . . 13
E. Distribution / access issues . . . 13
F. Script connection problems . . . 14
G. Diagnostics quick reference . . . 14
H. When to escalate . . . 14
7. Best Practices — Software License Usage Reporting & Cost Dashboards (Power BI) 14
8. Deliverables Summary — Software License Usage Reporting & Cost Dashboards (Power BI)
Documentation . . . 15
PowerShell scripts . . . 15
Named artifacts (reuse verbatim) . . . 16
What the buyer receives . . . 16
Outcome . . . 16
Appendix A — PowerShell Scripts 17

Executive Summary — Software License Usage Reporting & Cost Dashboards (Power BI)

License and spend data usually sits in exports nobody reads — so renewals surprise you, reharvesting savings go unclaimed, and leadership cannot see where the software money goes. This solution builds a repeatable license usage and cost reporting layer in Power BI: dashboards for deployment, usage, expiration timelines, reharvesting opportunities, and cost-efficiency metrics, fed from your license inventory and discovery/usage data and refreshed on a schedule.

This is a complete, do-it-the-repeatable-way package for standing up the license/spend reporting layer with Power BI Desktop, the Power BI service, and Excel Power Query, written for a Software Asset Management (SAM) analyst with no prior Power BI modeling experience.

What you get

- A plain-language Solution Overview of the license/spend reporting model, the data sources, and the KPI set.
- A Requirements Guide with Verify and Implement-if-missing for every prerequisite.
- An Implementation Guide of precise, ordered steps in Power BI Desktop, the Power BI service, and Excel.
- Five PowerShell scripts for reporting-data discovery, data-reconciliation validation, a consolidated license usage and cost report, a scheduled export helper that feeds Power BI, and a reporting-scope group builder.
- Testing and Troubleshooting guides for confident, accurate distribution.

What this delivers

A named KPI set — `License KPI Set – Software Assets – v1` (deployment rate, active usage, days-to-expiration, cost-per-seat, reharvesting savings) — modeled into one dataset, surfaced on a multi-page dashboard, refreshed on a schedule, and distributed to leadership and procurement only after a reconciliation gate confirms the numbers match the systems of record.

The standard deployment model

Assess to Configure to Pilot to Validate to Expand, with a reconciliation gate before broad distribution.

Outcome

License usage and spend made visible — deployment, expirations, reharvesting, and cost-efficiency dashboards that leadership and procurement actually use — refreshed on a schedule and proven on a pilot scope before you scale.

1. Solution Overview — Software License Usage Reporting & Cost Dashboards (Power BI)

What this reporting layer is

This is the license and spend reporting layer for Software Asset Management — distinct from device or endpoint health reporting. It takes three inputs (what you are entitled to, what is actually deployed or used, and what each entitlement costs and when it renews), models them into a single Power BI dataset, and presents a fixed set of license KPIs on dashboards that refresh on a schedule. The goal is to turn license data into decisions: which seats to reclaim, which renewals to right-size, and where the software spend actually goes.

Why it matters

Most organizations already pay for the data — entitlement records, M365/SaaS usage telemetry, discovery inventory, renewal dates — but it lives in disconnected exports. The result is predictable: renewals arrive as surprises, true-ups are over-bought, idle seats are never reharvested, and finance has no per-product cost view. Connecting those sources once and refreshing them on a schedule converts recurring manual export work into a living dashboard, so the SAM analyst spends time deciding rather than collecting. That is the only place in this document where the "why" is argued; the rest is "do this."

How it works

1. A defined KPI set (`License KPI Set – Software Assets – v1`) fixes exactly which numbers the

reporting layer produces, how each is calculated, and from which source.

1. License entitlement data (counts, renewal dates, unit cost) and usage/deployment data (active

users, installs, last activity) are connected as queries in one Power BI dataset.

1. A data model relates entitlements to usage by product, so each product shows entitled vs. deployed

vs. actively used — the basis for compliance, reharvesting, and cost-per-seat.

1. A multi-page dashboard surfaces the KPI set: a deployment/usage page, an expirations timeline, a

reharvesting opportunities page, and a cost-efficiency page.

1. Scheduled refresh keeps the dataset current; row-level reconciliation against the systems of record

gates broad distribution so leadership never sees numbers that disagree with the source.

1. The published report is distributed to a named audience (a leadership/procurement group) through the

Power BI service.

The license KPI set (what the dashboards report)

- Deployment rate — deployed (or assigned) divided by entitled, per product.
- Active usage rate — actively-used divided by deployed, per product (the reharvesting signal).
- Days-to-expiration — days until each contract/renewal date, for the expirations timeline.
- Cost-per-seat — annualized unit cost per entitlement, and per actively-used seat.

• Reharvesting savings — idle seats (deployed but not used) multiplied by unit cost = reclaimable spend.

Data sources (inputs to the model)

• License inventory / entitlements — counts, products, and unit cost from a SAM/ITAM tool, a controlled spreadsheet, or admin-center license exports.

• Usage / deployment data — M365/SaaS sign-in and activity reports, plus discovery/inventory (Intune, Configuration Manager, or a discovery tool) for installed software.

• Contracts / renewals — renewal dates and quantities from the entitlement system of record.

What you can control

• Which products and publishers are in scope (the reporting scope).

• The KPI definitions and thresholds (for example, the "idle" window for reharvesting).

• The refresh cadence (daily or weekly) and the distribution audience.

• The reconciliation tolerance that must pass before broad distribution.

Limitations (be honest with buyers)

• The dashboard is only as accurate as its inputs; a missing usage source produces blind spots, which the reconciliation gate is designed to catch.

• Scheduled refresh of cloud-to-on-prem sources may require an on-premises data gateway (called out in requirements).

• SaaS "usage" definitions vary by publisher; the KPI set documents the definition used per product.

• This layer reports and recommends; it does not reclaim seats or renew contracts — those run through the intake process.

The standard deployment model

> *Define the KPI set, connect license and usage data, build the dashboards, set scheduled refresh and distribution, validate the numbers, then pilot, validate, and expand. Pattern: Assess to Configure to Pilot to Validate to Expand, with a reconciliation gate before broad distribution.*

2. Requirements Guide — Software License Usage Reporting & Cost Dashboards (Power BI)

Verify every requirement before building. Each entry: Description · Why · Verify · Implement-if-missing. Category-wide prerequisites are in `global-requirements-asset-mgt.md`.

2.1 Power BI Desktop

Description: A current Power BI Desktop install on the SAM analyst's workstation. Why: The dataset, model, and dashboard pages are authored in Power BI Desktop before publishing. Verify: Power BI Desktop opens and Help shows a current version; it can create a blank report. Implement: Install Power BI Desktop from the Microsoft Store or the standalone installer.

2.2 Power BI service workspace

Description: A Power BI service workspace to publish the dataset and report into, plus rights to set scheduled refresh. Why: Scheduled refresh and distribution to leadership/procurement happen in the service, not Desktop. Verify: Sign in at `https://app.powerbi.com` and confirm you can open or create a workspace. Implement: Provision a Power BI (Pro or Premium Per User) license and create the workspace `License Reporting – Software Assets – Workspace`.

2.3 Excel with Power Query

Description: A current Excel with Power Query (Get & Transform) for staging and quick reconciliation. Why: License/contract exports are often cleaned and reconciled in Excel Power Query before modeling. Verify: In Excel, Data > Get Data is present (Power Query). Implement: Install a current Microsoft 365 Excel; Power Query is built in.

2.4 Access to license entitlement data

Description: A reachable source of entitlement counts, products, unit cost, and renewal dates (global G3). Why: The deployment, cost-per-seat, and expirations KPIs are all calculated against entitlements. Verify: You can open the entitlement source and read the count and renewal date for a sample product; run `LICRPT-Discovery.ps1` to confirm M365 license counts. Implement: Stand up the entitlement record (per global G3) and export current quantities, unit costs, and renewal dates.

2.5 Access to usage / deployment data

Description: A reachable source of actual usage and deployment — M365/SaaS activity reports and discovery inventory (global G4). Why: Active usage and reharvesting KPIs need real consumption, not just assignment. Verify: You can export a usage or installed-software report for a sample of users/devices; run `LICRPT-Discovery.ps1` to confirm M365 usage reports are reachable. Implement: Enable M365 usage reporting and Intune/Configuration Manager (or discovery-tool) inventory per global G4.

2.6 Microsoft Graph PowerShell (for the scripts)

Description: The Microsoft Graph PowerShell SDK on the workstation running the scripts. Why: The discovery, validation, report, and scheduled-export scripts read license and usage data through Microsoft Graph. Verify: `Get-Module -ListAvailable Microsoft.Graph.Authentication` returns a version. Implement: Run `Install-Module Microsoft.Graph -Scope CurrentUser`.

2.7 Read scopes for license & usage data

Description: Delegated/least-privilege read scopes for organization licensing and usage reports. Why: Reading subscribed SKUs and usage detail requires consent to specific Graph scopes. Verify: `Connect-MgGraph` succeeds with `Organization.Read.All`, `Reports.Read.All`, and

`User.Read.All`. Implement: Have an administrator consent the read scopes (least privilege; no write scopes are needed).

2.8 On-premises data gateway (only if a source is on-prem)

Description: An on-premises data gateway when a data source (a file share, an on-prem SQL/SAM database) is not cloud-reachable. Why: Scheduled refresh in the service can only reach on-prem sources through a gateway. Verify: In the service, Settings > Manage gateways shows a registered, online gateway covering the source. Implement: Install and register the standard on-premises data gateway; bind the dataset's source to it.

2.9 A reporting/distribution destination

Description: A defined audience and channel for the published report (an Entra group + the workspace/app). Why: Distribution targets a named leadership/procurement group, not individuals. Verify: The group `License Reporting – Leadership & Procurement – Audience` exists and members are correct. Implement: Create the audience group (or build it with `LICRPT-Set-ScopeTargeting.ps1`) and grant it report access.

Checklist

- Power BI Desktop installed
- Power BI service workspace `License Reporting – Software Assets – Workspace` created
- Excel with Power Query available
- License entitlement data reachable — `LICRPT-Discovery.ps1`
- Usage / deployment data reachable — `LICRPT-Discovery.ps1`
- Microsoft Graph PowerShell SDK installed
- Read scopes consented (`Organization.Read.All`, `Reports.Read.All`, `User.Read.All`)
- On-premises data gateway (only if a source is on-prem)
- Audience group `License Reporting – Leadership & Procurement – Audience` created

3. Implementation Guide — Software License Usage Reporting & Cost Dashboards (Power BI)

Follow these steps in order. Authoring is in Power BI Desktop; publishing, refresh, and distribution are in the Power BI service (`https://app.powerbi.com`). Excel Power Query is used to stage the license/contract export. Names in backticks are artifact names — reuse them verbatim.

> *Model: define the KPI set, connect license and usage data, build the dashboard pages, set scheduled refresh and distribution, then pilot, validate (reconciliation gate), and expand.*

Step 1 — Define and record the KPI set

1. Create a one-page definition for `License KPI Set – Software Assets – v1` listing each KPI, its

formula, and its source:

- Deployment rate = `Deployed / Entitled` per product.
- Active usage rate = `ActivelyUsed / Deployed` per product (define the active window, for example 30 days).
- Days-to-expiration = `RenewalDate - Today` per contract.
- Cost-per-seat = `AnnualUnitCost` and `AnnualUnitCost x Entitled / ActivelyUsed`.
- Reharvesting savings = `(Deployed - ActivelyUsed) x AnnualUnitCost` per product.

1. Save it beside the report; every later page implements exactly these definitions.

Step 2 — Stage the license/entitlement data (Excel Power Query)

1. Export entitlements (product, entitled quantity, annual unit cost, renewal date, publisher) to a file.
2. In Excel: Data > Get Data > From File, load the export, and in the Power Query Editor set types

(quantity and cost as numbers, renewal date as date) and trim/clean product names.

1. Name the query `License Entitlements – Software Assets – Source`. Close & Load to a table; save the workbook to the share or workspace location.

Step 3 — Generate the usage/deployment extract

1. Run `LICRPT-Export-Scheduled.ps1` to produce the usage/license extract CSV (a single dry run by

default; add the run switch to write the file). It writes subscribed-SKU counts and per-user activity from Microsoft Graph.

1. Confirm the output CSV contains product, deployed/assigned count, and last-activity columns. Place it in the same location as Step 2.

Step 4 — Build the dataset in Power BI Desktop

1. Open Power BI Desktop. Home > Get data: connect the `License Entitlements – Software Assets – Source`

table and the Step 3 usage CSV.

1. In Transform data (Power Query), align the product key in both queries (same casing/spelling) and set data types. Apply.
2. Model view: create a relationship from entitlements to usage on the product key (one-to-one or

one-to-many as your data dictates). Name the model file `License Usage & Cost – Software Assets – Dataset`.

Step 5 — Author the KPI measures (DAX)

1. New measure for each KPI in Step 1, for example:

• `Deployment Rate = DIVIDE(SUM(Usage[Deployed]), SUM(Entitlements[Entitled]))`

• `Reharvest Savings = SUMX(VALUES(Entitlements[Product]), (CALCULATE(SUM(Usage[Deployed])) - CALCULATE(SUM(Usage[ActivelyUsed]))) * CALCULATE(MAX(Entitlements[AnnualUnitCost])))`

1. Add a `Days To Expiration = DATEDIFF(TODAY(), MAX(Entitlements[RenewalDate]), DAY)` measure.

2. Keep every measure traceable to a Step 1 definition.

Step 6 — Build the dashboard pages

1. Add four report pages, named verbatim:

• `Deployment & Usage – Page` — a per-product table of entitled/deployed/used plus the deployment- and usage-rate cards.

• `Expirations Timeline – Page` — a timeline/bar of days-to-expiration, sorted ascending (soonest first).

• `Reharvesting Opportunities – Page` — products ranked by `Reharvest Savings` descending.

• `Cost Efficiency – Page` — cost-per-seat by product and total annualized spend.

1. Add a report-level slicer for publisher/product so the audience can filter to the reporting scope.

Step 7 — Publish to the service

1. Home > Publish; select the workspace `License Reporting – Software Assets – Workspace`.

2. Name the published report `License Usage & Cost – Software Assets – Report`.

Step 8 — Configure scheduled refresh

1. In the service, open the dataset > Settings.

2. Under Data source credentials, sign in for each source; if a source is on-prem, bind it to the gateway (requirement 2.8).

3. Under Scheduled refresh, enable it, set the cadence (daily or weekly), and name the schedule `License Refresh – Software Assets – Daily`. Add a failure-notification address.

Step 9 — Pilot on a single product family

1. Limit the slicer/scope to one well-understood product family (global G7 pilot scope).

2. Run `LICRPT-Report.ps1` to produce the same KPI numbers from source data independently of Power BI.

Step 10 — Validate (reconciliation gate)

1. Run `LICRPT-Validation.ps1` to reconcile the dataset's entitlement and usage counts against the systems of record; confirm the variance is within tolerance.

1. Compare the dashboard's pilot numbers to the `LICRPT-Report.ps1` output and to the entitlement system. Do not distribute broadly until they match within tolerance. Complete the Testing Guide.

Step 11 — Distribute and expand

1. After the gate passes, grant the report (or a published app) to the audience group `License Reporting – Leadership & Procurement – Audience` (build/update it with `LICRPT-Set-ScopeTargeting.ps1`).

1. Expand the reporting scope to the remaining products/publishers; re-run Steps 9 to 10 for each new scope before widening distribution.

Order of operations

```
Define KPI set -> stage entitlements (Excel) -> generate usage extract ->
build dataset -> author measures -> build pages -> publish -> scheduled
refresh -> pilot -> reconciliation gate -> distribute & expand
```

5. Testing Guide — Software License Usage Reporting & Cost Dashboards (Power BI)

Run after the pilot dataset publishes (Implementation Step 9) and after each refresh cycle or scope expansion.

Test 1 — Sources connect and refresh

Do: In the service, open the dataset `License Usage & Cost – Software Assets – Dataset` and choose Refresh now. Expected: The manual refresh completes with no credential or gateway errors.

Test 2 — KPI set matches definitions

Do: On each page, confirm the displayed measures match `License KPI Set – Software Assets – v1` (formula and source). Expected: Deployment rate, active usage rate, days-to-expiration, cost-per-seat, and reharvest savings are present and calculated as defined.

Test 3 — Numbers reconcile to source (the gate)

Do: Run `LICRPT-Report.ps1` and `LICRPT-Validation.ps1`; compare entitled and deployed counts to the dashboard for the pilot product family. Expected: Variance is within the agreed tolerance; `LICRPT-Validation.ps1` reports PASS.

Test 4 — Expirations timeline is correct

Do: Open `Expirations Timeline – Page`; check the soonest renewal against the entitlement system. Expected: Days-to-expiration is accurate and sorted soonest-first.

Test 5 — Reharvesting opportunities are credible

Do: Open `Reharvesting Opportunities – Page`; spot-check one product's idle-seat count against usage data. Expected: `(Deployed - ActivelyUsed)` and the savings figure match the underlying usage extract.

Test 6 — Scheduled refresh runs unattended

Do: Confirm `License Refresh – Software Assets – Daily` ran on schedule (dataset > Refresh history). Expected: The most recent scheduled refresh shows Completed; failure notifications are configured.

Test 7 — Distribution reaches the right audience

Do: Confirm only `License Reporting – Leadership & Procurement – Audience` members can open the report. Expected: Intended members have access; others do not.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1, 2, 3, 4, 5 pass for the pilot product family; reconciliation variance within tolerance
Distribution pass	Tests 6 and 7 pass; scheduled refresh green and audience access correct before broad distribution

6. Troubleshooting Guide — Software License Usage Reporting & Cost Dashboards (Power BI)

Use `LICRPT-Discovery.ps1` to confirm sources are reachable, `LICRPT-Validation.ps1` to find where the numbers disagree with the source, and `LICRPT-Report.ps1` to produce KPI numbers independently of Power BI for comparison.

A. Scheduled refresh fails

Symptom	Cause	Remediation
Refresh error on a cloud source	Expired/incorrect data source credentials	Dataset > Settings > Data source credentials: sign in again
Refresh error on an on-prem source	No gateway, or gateway offline	Install/register the on-premises data gateway and bind the source (requirement 2.8)

Symptom	Cause	Remediation
Refresh times out	Query too heavy / full reload each time	Reduce columns in Power Query; filter to the reporting scope; consider incremental refresh

B. Numbers do not reconcile (gate fails)

Symptom	Cause	Remediation
Dashboard count differs from source	Product key mismatch between entitlement and usage queries	Align product key casing/spelling in Power Query (Implementation Step 4)
Deployed greater than entitled looks wrong	Stale entitlement export or double-counted assignments	Re-export entitlements; de-duplicate usage; run `LICRPT-Validation.ps1` for the per-product variance
Usage looks too low	Active window too short, or a usage source missing	Confirm the active-usage window in the KPI set; verify the usage source via `LICRPT-Discovery.ps1`

C. KPI values look wrong

Symptom	Cause	Remediation
Rates show blank or error	DIVIDE denominator is zero/blank	Use DIVIDE (handles zero) and confirm both source columns load
Cost-per-seat inconsistent	Mixed cost bases (annual vs. monthly vs. list)	Normalize to annual unit cost in Power Query; document the basis (Best Practices)
Savings overstated	Idle definition too aggressive	Revisit the active-usage window in `License KPI Set – Software Assets – v1`

D. Expirations timeline empty or wrong

- Confirm renewal dates loaded as a Date type (not text) in Power Query, and that

`Days To Expiration` uses TODAY(); re-sort the page ascending by days-to-expiration.

E. Distribution / access issues

- A leadership member cannot open the report: confirm they are in

`License Reporting – Leadership & Procurement – Audience` and the workspace/app grants that group access; rebuild the group with `LICRPT-Set-ScopeTargeting.ps1`.

F. Script connection problems

- `Connect-MgGraph` fails or returns no data: confirm the read scopes

(`Organization.Read.All`, `Reports.Read.All`, `User.Read.All`) are consented (requirement 2.7) and that the Microsoft Graph SDK is installed (requirement 2.6).

- Usage report returns concealed names: tenant privacy setting masks user names in reports; have an admin disable concealment or report at the product/aggregate level.

G. Diagnostics quick reference

```
Get-MgContext                                    # confirm connected account and scopes
Get-MgSubscribedSku | Select SkuPartNumber, ConsumedUnits, PrepaidUnits   # entitlement vs. consumed
.\LICRPT-Discovery.ps1                           # list reachable license/usage sources
.\LICRPT-Validation.ps1                          # per-product reconciliation variance
```

H. When to escalate

If sources are reachable, scopes are consented, the product key is aligned, and `LICRPT-Validation.ps1` still reports a variance outside tolerance, capture the `LICRPT-Report.ps1` output alongside the entitlement system export and review the source data with the entitlement and procurement owners (global G6) before distributing.

7. Best Practices — Software License Usage Reporting & Cost Dashboards (Power BI)

- Fix the KPI set first. Lock `License KPI Set – Software Assets – v1` (formula and source per KPI)

before building anything; every page implements it. This is the single most important practice — it prevents two pages from defining "usage" two ways.

- Reconcile before you distribute. Always pass the reconciliation gate (`LICRPT-Validation.ps1` plus a

manual spot-check) on a pilot scope before leadership or procurement sees a number; a wrong number loses the dashboard's credibility permanently.

- Separate entitled, deployed, and actively used. Report all three per product; the gap between

deployed and actively used is the reharvesting opportunity, and the gap between entitled and deployed is the compliance/true-up signal.

- Name every artifact with `<Purpose> – <Scope> – <Stage>` and reuse the names verbatim across the

dataset, pages, refresh schedule, and audience group, so the whole reporting layer is traceable.

- Sort expirations soonest-first. The expirations page exists to prevent renewal surprises; ascending

days-to-expiration is what makes it actionable.

- Define "idle" explicitly. Document the active-usage window (for example 30 or 90 days) in the KPI set

so reharvesting numbers are defensible to the seat owner.

• Schedule refresh to match decisions, not vanity. Daily for fast-moving SaaS, weekly for stable perpetual products; always set a failure-notification address so a stale dashboard does not pass unnoticed.

• Keep read-only. The reporting layer reads license and usage data; reclaiming seats and renewing contracts run through the intake process (global G6), never from the dashboard.

• Document the cost basis. Record whether unit cost is annualized, list, or negotiated, so cost-per-seat and savings figures are comparable across products and audits.

• Use a gateway for on-prem sources. Bind on-prem files/databases to the on-premises data gateway so scheduled refresh stays unattended instead of failing silently.

8. Deliverables Summary — Software License Usage Reporting & Cost Dashboards (Power BI)

Documentation

Document	Purpose
Executive Summary	Business case and what is in the box
1. Solution Overview	The license/spend reporting model, data sources, the KPI set, and the refresh cycle
2. Requirements Guide	Every prerequisite with Verify and Implement-if-missing
Global Requirements (Asset Mgt)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps in Power BI Desktop, the service, and Excel
5. Testing Guide	Validation procedures with pass/fail gates, including the reconciliation gate
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`LICRPT-Discovery.ps1`	Reports available license (subscribed SKU) and usage report sources via Microsoft Graph; confirms reporting readiness (read-only)

Script	What it does
`LICRPT-Validation.ps1`	Reconciles entitlement and usage counts against the source and reports variance vs. tolerance (read-only)
`LICRPT-Report.ps1`	Produces the license KPI set (deployment, usage, expirations, cost-per-seat, savings) independently of Power BI (read-only)
`LICRPT-Export-Scheduled.ps1`	Exports the license/usage extract that feeds the Power BI dataset; single dry run by default
`LICRPT-Set-ScopeTargeting.ps1`	Builds/updates the reporting-scope and audience groups for distribution

Named artifacts (reuse verbatim)

- KPI set: `License KPI Set – Software Assets – v1`
- Dataset: `License Usage & Cost – Software Assets – Dataset`
- Report: `License Usage & Cost – Software Assets – Report`
- Workspace: `License Reporting – Software Assets – Workspace`
- Dashboard pages: `Deployment & Usage – Page`, `Expirations Timeline – Page`, `Reharvesting Opportunities – Page`, `Cost Efficiency – Page`
- Refresh schedule: `License Refresh – Software Assets – Daily`
- Audience group: `License Reporting – Leadership & Procurement – Audience`

What the buyer receives

- This implementation guide as a Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process: Assess to Configure to Pilot to Validate to Expand, with a reconciliation gate before broad distribution.

Outcome

License usage and spend made visible — deployment, expirations, reharvesting, and cost-efficiency dashboards refreshed on a schedule and proven on a pilot scope before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

LICRPT-Discovery.ps1

Purpose: Reports the tenant's subscribed license SKUs (entitled vs. consumed) and confirms that the Microsoft 365 usage reports are reachable, so you know which products and usage signals are available to feed the Power BI license usage and cost reporting layer. Read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\LICRPT-Discovery.ps1 Requires the Microsoft Graph PowerShell SDK and the read scopes Organization.Read.All and Reports.Read.All. EXPECTED RESULTS A table of subscribed SKUs with prepaid (entitled) and consumed (assigned) units, plus a confirmation that the usage report endpoint returned data. .NOTES Software Asset Management — License Reporting. Read-only against Microsoft Graph. #> [CmdletBinding()] param()

How to run: powershell -ExecutionPolicy Bypass -File .\LICRPT-Discovery.ps1 Requires the Microsoft Graph PowerShell SDK and the read scopes Organization.Read.All and Reports.Read.All. EXPECTED RESULTS A table of subscribed SKUs with prepaid (entitled) and consumed (assigned) units, plus a confirmation that the usage report endpoint returned data. .NOTES Software Asset Management — License Reporting. Read-only against Microsoft Graph. #> [CmdletBinding()] param()

LICRPT-Export-Scheduled.ps1

Purpose: Produces the recurring extract the Power BI dataset 'License Usage & Cost - Software Assets - Dataset' consumes: per-SKU entitled/assigned counts (subscribed SKUs) plus per-user license activity (last activity date) from the Microsoft 365 usage reports. Intended to be run on a schedule (Task Scheduler) so the dashboard always has a fresh source file. HOW TO RUN Dry run (default — reports what it WOULD write, writes nothing): powershell -ExecutionPolicy Bypass -File .\LICRPT-Export-Scheduled.ps1 Real export (writes the CSVs): powershell -ExecutionPolicy Bypass -File .\LICRPT-Export-Scheduled.ps1 -Run -OutDir C:\LicReporting Requires the Microsoft Graph SDK and the read scopes Organization.Read.All and Reports.Read.All. EXPECTED RESULTS Dry run: counts of SKUs and usage rows that would be exported, and the target file paths. Real run: licrpt-skus.csv and licrpt-usage.csv written to -OutDir for Power BI to refresh from. .NOTES Software Asset Management — License Reporting. Defaults to a single dry run; -Run performs the export. #> [CmdletBinding()] param([switch]$Run, [string]$OutDir = "$env:TEMP\LicReporting", [ValidateSet('D7','D30','D90','D180')][string]$Period = 'D30')

How to run: Dry run (default — reports what it WOULD write, writes nothing): powershell -ExecutionPolicy Bypass -File .\LICRPT-Export-Scheduled.ps1 Real export (writes the CSVs): powershell -ExecutionPolicy Bypass -File .\LICRPT-Export-Scheduled.ps1 -Run -OutDir C:\LicReporting Requires the Microsoft Graph SDK and the read scopes Organization.Read.All and Reports.Read.All. EXPECTED RESULTS Dry run: counts of SKUs and usage rows that would be exported, and the target file paths. Real run: licrpt-skus.csv and licrpt-usage.csv written to -OutDir for Power BI to refresh from. .NOTES Software Asset Management — License Reporting. Defaults to a single dry run; -Run performs the export. #> [CmdletBinding()] param([switch]$Run, [string]$OutDir = "$env:TEMP\LicReporting", [ValidateSet('D7','D30','D90','D180')][string]$Period = 'D30')

LICRPT-Report.ps1

Purpose: Builds the License KPI Set - Software Assets - v1 (deployment rate, active usage rate, cost-per-seat, and reharvesting savings) from Microsoft Graph subscribed-SKU data joined to an optional cost/renewal baseline, so you can verify the Power BI dashboard against an independent source. Read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\LICRPT-Report.ps1 -CostCsv C:\Temp\costs.csv -CsvPath C:\Temp\licrpt-kpis.csv Optional CostCsv columns: SkuPartNumber, AnnualUnitCost, RenewalDate. Requires the Microsoft Graph SDK and Organization.Read.All. EXPECTED RESULTS A per-SKU table of entitled, assigned, deployment rate, cost-per-seat, days-to-expiration, and reharvesting savings, with an optional CSV export for comparison against the dashboard. .NOTES Software Asset Management — License Reporting. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CostCsv, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\LICRPT-Report.ps1 -CostCsv C:\Temp\costs.csv -CsvPath C:\Temp\licrpt-kpis.csv Optional CostCsv columns: SkuPartNumber, AnnualUnitCost, RenewalDate. Requires the Microsoft Graph SDK and Organization.Read.All. EXPECTED RESULTS A per-SKU table of entitled, assigned, deployment rate, cost-per-seat, days-to-expiration, and reharvesting savings, with an optional CSV export for comparison against the dashboard. .NOTES Software Asset Management — License Reporting. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CostCsv, [string]$CsvPath)

LICRPT-Set-ScopeTargeting.ps1

Purpose: Creates (if missing) and populates the Entra security groups used by the reporting layer: the distribution audience 'License Reporting - Leadership & Procurement - Audience' that the published Power BI report is shared with. Members are added from a CSV of user principal names. This is the only script that writes (group membership); discovery/validation/report are read-only. HOW TO RUN Preview (default — shows what WOULD change, writes nothing): powershell -ExecutionPolicy Bypass -File .\LICRPT-Set-ScopeTargeting.ps1 -MembersCsv C:\Temp\audience.csv Apply the changes: powershell -ExecutionPolicy Bypass -File .\LICRPT-Set-ScopeTargeting.ps1 -MembersCsv C:\Temp\audience.csv -Apply MembersCsv must have a column: UserPrincipalName. Requires the Microsoft Graph SDK and the scopes Group.ReadWrite.All and User.Read.All. EXPECTED RESULTS Preview: the target group and the members that would be added. Apply: the group exists and the listed members are added; the group is then granted report access in Power BI. .NOTES Software Asset Management — License Reporting. Writes group membership only when -Apply is supplied. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$MembersCsv, [string]$GroupName = 'License Reporting - Leadership & Procurement - Audience', [switch]$Apply)

How to run: Preview (default — shows what WOULD change, writes nothing): powershell -ExecutionPolicy Bypass -File .\LICRPT-Set-ScopeTargeting.ps1 -MembersCsv C:\Temp\audience.csv Apply the changes: powershell -ExecutionPolicy Bypass -File .\LICRPT-Set-ScopeTargeting.ps1 -MembersCsv C:\Temp\audience.csv -Apply MembersCsv must have a column: UserPrincipalName. Requires the Microsoft Graph SDK and the scopes Group.ReadWrite.All and User.Read.All. EXPECTED RESULTS Preview: the target group and the members that would be added. Apply: the group exists and the listed members are added; the group is then granted report access in Power BI. .NOTES Software Asset Management — License Reporting. Writes group membership only when -Apply is supplied. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$MembersCsv, [string]$GroupName = 'License Reporting - Leadership & Procurement - Audience', [switch]$Apply)

LICRPT-Validation.ps1

Purpose: Compares the entitlement (prepaid) and assigned (consumed) counts that Microsoft Graph reports for each subscribed SKU against a baseline export from your entitlement system of record, and reports the per-product variance versus an allowed tolerance. This is the gate that must pass before the Power BI dashboard is distributed broadly. Read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\LICRPT-Validation.ps1 -BaselineCsv C:\Temp\entitlements.csv -TolerancePct 2 The baseline CSV must have columns: SkuPartNumber, Entitled, Assigned. Requires the Microsoft Graph SDK and Organization.Read.All. EXPECTED RESULTS A per-SKU table of source vs. Graph counts with variance %, and an overall PASS or REVIEW result against the tolerance. Exit code 0 = PASS, 1 = REVIEW. .NOTES Software Asset Management — License Reporting. Read-only against Microsoft Graph. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$BaselineCsv, [double]$TolerancePct = 2)

How to run: powershell -ExecutionPolicy Bypass -File .\LICRPT-Validation.ps1 -BaselineCsv C:\Temp\entitlements.csv -TolerancePct 2 The baseline CSV must have columns: SkuPartNumber, Entitled, Assigned. Requires the Microsoft Graph SDK and Organization.Read.All. EXPECTED RESULTS A per-SKU table of source vs. Graph counts with variance %, and an overall PASS or REVIEW result against the tolerance. Exit code 0 = PASS, 1 = REVIEW. .NOTES Software Asset Management — License Reporting. Read-only against Microsoft Graph. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$BaselineCsv, [double]$TolerancePct = 2)